

Computing Constrained Longest Common Subsequence for Degenerate Strings using Finite Automata

Costas Iliopoulos^{1,3}, M. Sohel Rahman^{1,3},
Michal Voráček^{2,4}, and Ladislav Vagner^{2,4}

¹ Algorithm Design Group
Department of Computer Science
King's College London
Strand, London WC2R 2LS, England
<http://www.dcs.kcl.ac.uk/adg>

² Department of Computer Science & Engineering
Czech Technical University
Czech Republic

³ {csi, sohel}@dcs.kcl.ac.uk

⁴ {voracem, xvagner}@fel.cvut.cz

Abstract. In this paper, we study the *Constrained Longest Common Subsequence* (CLCS) problem for *degenerate strings*, which is a new variant of the classical Longest Common Subsequence problem. In CLCS, the computed longest common subsequence must also be a supersequence of a third given string. We present a finite automata based algorithm to solve the problem. The algorithm requires $\mathcal{O}(|\Sigma|nmr)$ time in the worst case, where $|\Sigma|$ is size of alphabet, n, m and r are the lengths of the three given degenerate strings. To the best of our knowledge, this is the first attempt to solve the CLCS problem for degenerate strings.

1 Introduction

In this paper, we present a finite automata based algorithm for solving the *constrained longest common subsequence* problem for *degenerate strings*. A *string* is a sequence of symbols from a given alphabet Σ . A *subsequence* u of a string x is obtained by deleting some characters from u (not necessarily contiguous). Given two strings x and y , u is a *common subsequence* of x and y , if u is a subsequence of both x and y . And, u is a *longest common subsequence* (LCS) of x and y , if it is the longest among all such subsequences. The LCS problem is that of computing the longest common subsequence of two given strings. The LCS problem is one of the classical and well-studied problems in computer science having extensive applications in diverse areas. This is the reason that the LCS problem and variants thereof have been the focus of extensive research in the computer science literature since long.

In this paper, we study a newer variant of the classic LCS problem, namely the Constrained LCS problem (CLCS). In CLCS, the computed longest common subsequence must also be a supersequence of a third given string, say z [4, 3, 2]. This problem itself is interesting from the combinatorial point of view but its main motivation comes from bioinformatics: in the computation of the homology of two biological sequences it is important to take into account a common specific or putative structure [4].

The CLCS problem was introduced, quite recently, by Tsai in [4], where an algorithm was presented solving the problem in $\mathcal{O}(n^2m^2r)$ time complexity. Here $n = |x|, m = |y|$ and $r = |z|$. Later, Chin et al. [1] and independently, Arslan and Egecioğlu [2, 3] presented improved algorithm with $\mathcal{O}(nmr)$ time and space complexity.

In this paper, we consider the CLCS problem for degenerate strings, i.e. strings that may have a set of letters in each position. In our case, all the three input strings are degenerate. The motivation of our study comes from the fact that degenerate strings are extensively used in molecular biology to express polymorphism in DNA sequences, e.g. the polymorphism of protein coding regions caused by redundancy of the genetic code or polymorphism in binding site sequences of a family of genes.

The main contribution of this paper is as follows. We present an efficient algorithm based on finite automata to solve the CLCS problem for degenerate strings. Our approach utilizes two novel types of finite automata, namely *subsequence automaton for degenerate strings*⁵ and *supersequence automaton for degenerate strings*. We first present algorithms for the construction of the above two automata, both of which work in linear time with respect to length of the given degenerate string. Then we show how to use these two automata to solve CLCS problem for degenerate strings in $\mathcal{O}(|\Sigma|nmr)$ worst case time complexity. Note that, in DNA/RNA and protein sequences the cardinality of Σ is, respectively, 4 and 20 and hence can safely be assumed to be constant. Therefore, for biological sequences, our algorithm works in $\mathcal{O}(nmr)$ time, which matches the time complexity of the best-known algorithms [1–3] to solve CLCS for normal strings. To the best of our knowledge, this is the first attempt to solve the CLCS problem for degenerate strings.

The rest of the paper is organized as follows. In Section 2, we present the preliminary concepts and in Section 3, we present our main result, i.e. the algorithm to solve CLCS problem for degenerate strings. Finally, we conclude briefly in Section 4.

2 Preliminaries

An *alphabet* Σ is a nonempty finite set of symbols. A *string* over a given alphabet is a finite sequence of symbols. The *empty string* is denoted by ε . The set of all strings over an alphabet Σ (including empty string ε) is denoted by Σ^* .

Definition 1. A string $\tilde{x} = \tilde{x}_1\tilde{x}_2\ldots\tilde{x}_n$ is said to be *degenerate*, if it is built over the potential $2^{|\Sigma|} - 1$ non-empty sets of letters belonging to Σ . We say that $\tilde{x}_i \in \{\mathcal{P}(\Sigma) \setminus \emptyset\} = \mathcal{P}^+(\Sigma)$ and $|\tilde{x}_i|$ denotes the cardinality of $\tilde{x}_i$.

In what follows, the set containing the letters a and c will be denoted by $[a, c]$ and the singleton $[c]$ will be simply denoted by c for ease of reading. Also, we use the following convention: we use normal letters like x to denote normal strings. The same letter may be used to denote a degenerate string if used in the following way: $\tilde{x}$.

Definition 2. The language represented by degenerate string $\tilde{x}$ is the set $\mathcal{L}(\tilde{x}) = \{u \mid u = u_1u_2\ldots u_n, u_j \in \tilde{x}_j, 1 \leq j \leq n, u \in \Sigma^*\}$. A string u is said to be an element of degenerate string $\tilde{x}$, denoted $u \in \tilde{x}$, if it is an element of language represented by $\tilde{x}$.

We define the concatenation operation on the set of degenerate strings in the usual way: if $\tilde{x}$ and $\tilde{y}$ are degenerate strings over Σ , then the concatenation of these strings is $\tilde{x}\tilde{y}$. The length $|\tilde{x}|$ of degenerate string $\tilde{x}$ is the number of its sets (of letters). Note that, there may exist singleton sets. A degenerate string $\tilde{v}$ is a *factor* (resp. *prefix*, *suffix*) of a degenerate string $\tilde{x}$ if $\tilde{x} = \tilde{u}\tilde{v}\tilde{w}$ (resp. $\tilde{x} = \tilde{v}\tilde{w}$, $\tilde{x} = \tilde{u}\tilde{v}$).

⁵ This can be seen as an extension of subsequence automaton for (normal) strings also known as *direct acyclic subsequence graph (DASG)* in the literature. A finite automaton accepting all subsequences of a given string was first mentioned by Hébrard and Crochemore [7]. Baeza-Yates proposed its right to left construction [6] and Troníček and Melichar presented its left to right construction [5].

Definition 3. A (normal) string $v = v_1 \dots v_\ell$ of length ℓ is said to occur in a degenerate string $\tilde{x} = \tilde{x}_1 \tilde{x}_2 \dots \tilde{x}_n$ at position $i, 1 \leq i \leq n$ if and only if $v_j \in \tilde{x}_{i-\ell+j}$ for all $1 \leq j \leq \ell$. The position of the first occurrence of v in $\tilde{x}$ is denoted by $\text{foccur}(v, \tilde{x})$.

Definition 4 (Sets of Subsequences of Degenrate Strings). For degenerate strings $\tilde{x} = \tilde{x}_1 \tilde{x}_2 \dots \tilde{x}_n$, $\tilde{y} = \tilde{y}_1 \tilde{y}_2 \dots \tilde{y}_m$ and $\tilde{z} = \tilde{z}_1 \tilde{z}_2 \dots \tilde{z}_r$ over an alphabet Σ we define followings sets:

1. Set of all subsequences of degenerate string $\tilde{x}$:

$$\begin{aligned} \text{Sub}(\tilde{x}) = \{ u \mid u \in \tilde{x}_{i_1} \tilde{x}_{i_2} \dots \tilde{x}_{i_m} \wedge \tilde{x} = & \tilde{x}_1 \tilde{x}_2 \dots \tilde{x}_{i_1-1} \tilde{x}_{i_1} \\ & \tilde{x}_{i_1+1} \dots \tilde{x}_{i_2-1} \tilde{x}_{i_2} \\ & \tilde{x}_{i_2+1} \dots \tilde{x}_{i_m-1} \tilde{x}_{i_m} \\ & \tilde{x}_{i_m+1} \dots \tilde{x}_n, \\ & \tilde{x}_i \in \mathcal{P}^+(\Sigma), 1 \leq i \leq n, 1 \leq i_1 < i_2 < \dots < i_m \leq n, m \geq 0 \} \end{aligned}$$

2. Set of all common subsequences of $\tilde{x}$ and $\tilde{y}$:

$$\text{CSub}(\tilde{x}, \tilde{y}) = \{ u \mid u \in \text{Sub}(\tilde{x}) \wedge u \in \text{Sub}(\tilde{y}) \}$$

3. Set of all constrained common subsequences of $\tilde{x}$ and $\tilde{y}$ with respect to $\tilde{z}$:

$$\text{CCSub}(\tilde{x}, \tilde{y}, \tilde{z}) = \{ u \mid u \in \text{CSub}(\tilde{x}, \tilde{y}) \wedge \exists v : v \in \tilde{z} \wedge v \in \text{Sub}(u) \}$$

4. Set of all longest common subsequences of $\tilde{x}$ and $\tilde{y}$:

$$\text{LCSub}(\tilde{x}, \tilde{y}) = \{ u \mid u \in \text{CSub}(\tilde{x}, \tilde{y}) \wedge \forall v \in \text{CSub}(\tilde{x}, \tilde{y}) \ |v| \leq |u| \}$$

5. Set of all constrained longest common subsequences of $\tilde{x}$ and $\tilde{y}$ with respect to $\tilde{z}$:

$$\text{CLCSub}(\tilde{x}, \tilde{y}, \tilde{z}) = \{ u \mid u \in \text{CCSub}(\tilde{x}, \tilde{y}, \tilde{z}) \wedge \forall v \in \text{CCSub}(\tilde{x}, \tilde{y}, \tilde{z}) \ |v| \leq |u| \}$$

6. Set of all supersequences of $\tilde{x}$:

$$\text{Super}(\tilde{x}) = \{ v \mid v \in \Sigma^* \wedge \exists u \in \tilde{x} : u \in \text{Sub}(v) \}$$

Definition 5 (Maximum Length Function). Let L be a finite language over an alphabet Σ . The Maximum length function, $\text{MaxLen} : \mathcal{P}(\Sigma^*) \mapsto \mathcal{P}(\Sigma^*)$, is defined for L as follows:

$$\text{MaxLen}(L) = \{ w \mid w \in L \wedge \forall v \in L : |v| \leq |w| \}$$

Since our algorithms are based on a finite automata approach, we give brief definitions to related concepts below. A nondeterministic finite automaton M is a quintuple $(Q, \Sigma, \delta, q_0, F)$, where: Q is a finite set of states, Σ is an input alphabet, δ is a mapping $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \mapsto \mathcal{P}(Q)$ called a state transition function, $I \subseteq Q$ is a set of initial states, and $F \subseteq Q$ is a set of final states. A deterministic finite automaton M is a special case of nondeterministic finite automaton such that transition mapping is a function $\delta : Q \times \Sigma \mapsto Q$ and there is only one initial state $q_0 \in Q$. The *left language* of state q of finite automaton M , denoted $\overleftarrow{\mathcal{L}}_M(q)$, is a set of strings for which there exists a sequence of transitions from the initial state to state q . *Language accepted* by finite automaton M , denoted $\mathcal{L}(M)$, is a set of words for which there exists sequence of transitions from the initial state to some of the final states. The *depth* of state q of acyclic finite automaton M is the length of the longest string in its left language.

We conclude this section by giving a formal definition of the CLCS problem for degenerate strings along with an example.

Problem “DCLCS” (CLCS Problem for Degenerate Strings). Given degenerate strings $\tilde{x}$, $\tilde{y}$ and $\tilde{z}$, the constrained longest common subsequence problem for $\tilde{x}$ and $\tilde{y}$ with respect to $\tilde{z}$ is to find a common subsequence v of $\tilde{x}$ and $\tilde{y}$ such that it contains as a subsequence some string $u \in \tilde{z}$ and v is the longest among such common subsequences.

Example 1. Suppose, we are given the degenerate strings $\tilde{x} = [a, f]b d d a a a$, $\tilde{y} = [a, c]b a [c, d] a a [d, f]$ and $\tilde{z} = b[c, d]d$. The table on the left shows an example of LCS of degenerate strings $\tilde{x}$ and $\tilde{y}$ and the table on the right shows an example of CLCS of degenerate strings $\tilde{x}$ and $\tilde{y}$ with respect to degenerate string $\tilde{z}$. Note that although the LCS is of length 5 the CLCS is of length 4 only.

$\tilde{x}$	$[a, f]$	b	d	d	a	a	a
$\tilde{y}$	$[a, c]$	b	a	$[c, d]$	a	a	$[d, f]$
LCS	a	b	d	a	a		

$\tilde{x}$	$[a, f]$	b	d	d	a	a	a
$\tilde{y}$	$[a, c]$	b	a	$[c, d]$	a	a	$[d, f]$
$\tilde{z}$		b	$[c, d]$	d			
CLCS	a	b	d	d			

3 Algorithm

In this section, we present the algorithm to solve Problem DCLCS. The algorithm is based on application of finite automata. The resulting algorithm works in $\mathcal{O}(|\Sigma|nmr)$ time and space. Our algorithm is based on the following lemma:

Lemma 1. Let $\tilde{x}$, $\tilde{y}$ and $\tilde{z}$ be degenerate strings over an alphabet Σ , then it holds:

$$CLCSub(\tilde{x}, \tilde{y}, \tilde{z}) = MaxLen(Sub(\tilde{x}) \cap Sub(\tilde{y}) \cap Super(\tilde{z})).$$

Proof. The lemma basically follows directly from the definitions of the sets involved in the Lemma. $\square$

Note that, all the sets considered in Lemma 1, form regular languages. This implies that we can represent each of them by a finite automaton. Therefore, all the strings of the (output) set can then be easily obtained by finding paths (e.g. by DFS traversal) in the transition diagram of the (output) automaton from the initial state to all final states. The string(s) are then given by spelling the labels of transitions on the paths. So, what we do is as follows. We first construct automata representing the sets $Sub(\tilde{x})$, $Sub(\tilde{y})$ and $Super(\tilde{z})$. Then, we perform the intersection operation followed by the selection of longest strings to get the desired automaton for $MaxLen(Sub(\tilde{x}) \cap Sub(\tilde{y}) \cap Super(\tilde{z}))$. The steps are formally presented in the form of Algorithm 1. In the following subsections we discuss the steps of Algorithm 1 in detail and analyze the running time of the algorithm. The following table shows the road-map of the rest of this section.

Section	Content	Applies to Algorithm 1
3.1	Subsequence Automaton for Degenerate Strings	Step 1
3.2	Finite Automaton for Intersection of Languages	Step 2 and 4
3.3	Supersequence Automaton for Degenerate Strings	Step 3
3.4	Maximum Length Strings Automaton	Step 5
3.5	Time Complexity Analysis	

procedure: CLCSUBSEQUENCE-AUTOMATON

input: Degenerate strings $\tilde{x}, \tilde{y}, \tilde{z}$

output: $M = (Q, \Sigma, \delta, q_0, F), \mathcal{L}(M) = CLCSub(\tilde{x}, \tilde{y}, \tilde{z})$

1. (a) For degenerate string $\tilde{x}$ construct automaton M_1 using Alg. 2, $M_1 \leftarrow \text{SUBSEQUENCE-AUTOMATON}(\tilde{x})$, $\mathcal{L}(M_1) = \text{Sub}(\tilde{x})$.
- (b) For degenerate string $\tilde{y}$ construct automaton M_2 using Alg. 2, $M_2 \leftarrow \text{SUBSEQUENCE-AUTOMATON}(\tilde{y})$, $\mathcal{L}(M_2) = \text{Sub}(\tilde{y})$.
2. For automata M_1 and M_2 construct automaton M_3 using Alg. 3, $M_3 \leftarrow \text{INTERSECTION}(M_1, M_2)$, $\mathcal{L}(M_3) = \mathcal{L}(M_1) \cap \mathcal{L}(M_2) = \text{Sub}(\tilde{x}) \cap \text{Sub}(\tilde{y})$.
3. For degenerate string $\tilde{z}$ construct automaton M_4 using Alg. 4, $M_4 \leftarrow \text{SUPERSEQUENCE-AUTOMATON}(\tilde{z})$, $\mathcal{L}(M_4) = \text{Super}(\tilde{z})$.
4. For automata M_3 and M_4 construct automaton M_5 using Alg. 3, $M_5 \leftarrow \text{INTERSECTION}(M_3, M_4)$, $\mathcal{L}(M_5) = \mathcal{L}(M_3) \cap \mathcal{L}(M_4) = ((\text{Sub}(\tilde{x}) \cap \text{Sub}(\tilde{y})) \cap \text{Super}(\tilde{z}))$.
5. Transform automaton M_5 into automaton M using Alg. 5, $M \leftarrow \text{MAXLEN-AUTOMATON}(M_5)$, $\mathcal{L}(M) = \text{MaxLen}(\mathcal{L}(M_5)) = \text{MaxLen}((\text{Sub}(\tilde{x}) \cap \text{Sub}(\tilde{y})) \cap \text{Super}(\tilde{z}))$.

Algorithm 1: Construction of deterministic finite automaton M accepting language $\mathcal{L}(M) = CLCSub(\tilde{x}, \tilde{y}, \tilde{z})$

3.1 Subsequence Automaton for Degenerate Strings

In this section, we present a novel type of finite automaton, referred to as the *subsequence automaton* (*SubA*) henceforth, which accepts the set of all subsequences of a given degenerate string. The subsequence automaton is deterministic, acyclic and minimal and its number of states and transitions are linear with respect to the length of the degenerate string.

The set of all subsequences of degenerate string $\tilde{x} = [\tilde{x}_{11}, \tilde{x}_{12}, \dots, \tilde{x}_{1|\tilde{x}_1|}] \dots \dots [\tilde{x}_{n1}, \tilde{x}_{n2}, \dots, \tilde{x}_{n|\tilde{x}_n|}]$ (see Def. 4) can be expressed by the regular expression:

$$R_S = (\tilde{x}_{11} + \tilde{x}_{12} + \dots + \tilde{x}_{1|\tilde{x}_1|} + \varepsilon) \dots (\tilde{x}_{n1} + \tilde{x}_{n2} + \dots + \tilde{x}_{n|\tilde{x}_n|} + \varepsilon).$$

The regular expression R_S can be directly used for the construction of a nondeterministic automaton accepting the set $\text{Sub}(\tilde{x})$ (see Fig. 3.2, automaton M'_1). Let us denote the automaton constructed in this way by M' . The deterministic automaton M obtained by determinization of automaton M' (see Fig.3.2, automaton M_1) is defined as follows:

$M = (Q, \Sigma, \delta, q_0, F)$, where $Q = F = \{q_0, q_1, q_2, \dots, q_n\}$ and $\delta(q_i, s) = q_j$, where $j = \min\{k \mid s \in \tilde{x}_k, i+1 \leq k \leq n\}$ if $\{k \mid s \in \tilde{x}_k, i+1 \leq k \leq n\} \neq \emptyset$ else $\delta(q_i, s)$ is undefined.

The deterministic subsequence automaton M as defined above can be constructed directly without the prior construction of nondeterministic automaton and its consequent determinization. In Appendix A, we provide an online algorithm working in linear time for its direct construction. The basic properties of the subsequence automaton and of the Algorithm 2 for its direct construction can be summarized in the form of following Lemmas.

Lemma 2. *Let $M = (Q, \Sigma, \delta, q_0, F)$ be an automaton built by Algorithm 2 on degenerate string $\tilde{x} = \tilde{x}_1 \tilde{x}_2 \dots \tilde{x}_n$. Then, for all $q \in Q$ and for all $u \in \Sigma^*$ it holds*

$$(u \in \overleftarrow{\mathcal{L}}_M(q) \wedge \text{depth}(q) = i) \Leftrightarrow (u \in \text{Sub}(\tilde{x}) \wedge \text{foccur}(u, \tilde{x}) = i)$$

Lemma 3. *Given a degenerate string $\tilde{x}$ of length n over an alphabet Σ , Algorithm 2 correctly constructs deterministic finite automaton M accepting language $\mathcal{L}(M) = \text{Sub}(\tilde{x})$ in*

$\mathcal{O}(n|\Sigma|)$ time using $\mathcal{O}(|\Sigma|)$ additional space. M is acyclic, minimal and has exactly $n + 1$ states and $\mathcal{O}(|\Sigma|n)$ transitions.

3.2 Finite Automaton for Intersection of Languages

In this section, we discuss the construction of an automaton for the intersection of languages. We use a variant of standard algorithm that creates only accessible states. This algorithm builds the resulting automaton by simultaneous traversal of the input automata. The algorithm can be found e.g. in [8] and it is also provided in Appendix B. The key properties of the resulting automaton and the algorithm are expressed by the following lemma:

Lemma 4. *Given deterministic finite automata M_1 and M_2 having n and m states, respectively, Algorithm 3 correctly constructs deterministic finite automaton M accepting language $\mathcal{L}(M) = \mathcal{L}(M_1) \cap \mathcal{L}(M_2)$ in $\mathcal{O}(|\Sigma|nm)$ time. M has at most nm states and at most $|\Sigma|nm$ transitions. Moreover, if M_1 (or M_2) is acyclic, then M is also acyclic.*

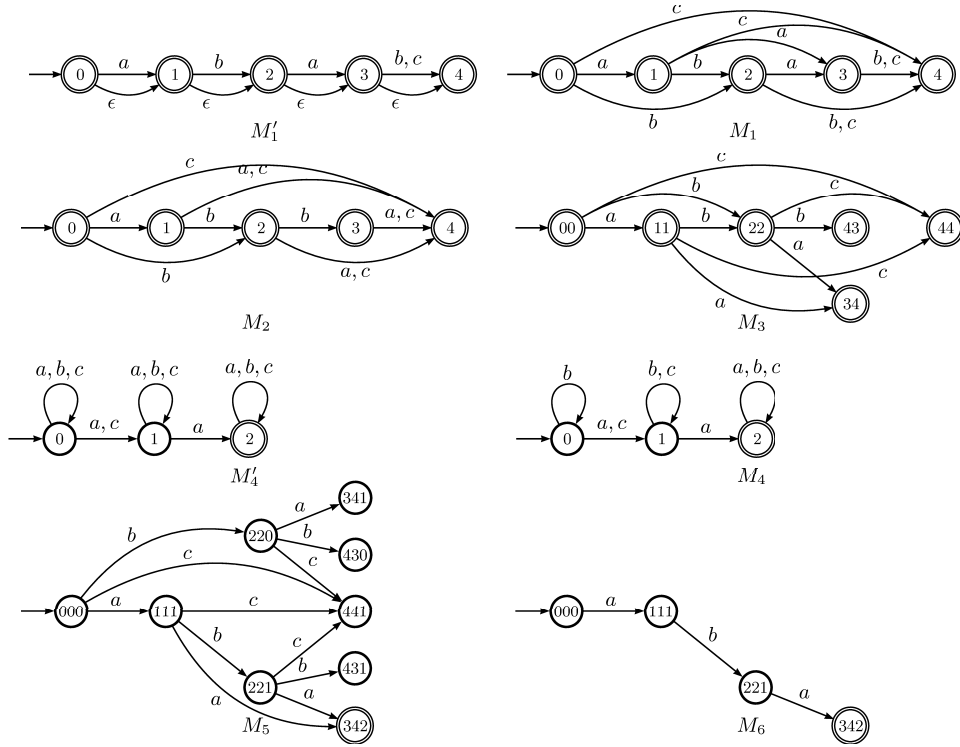


Fig. 1. Nondeterministic finite automaton M'_1 , M'_4 and deterministic finite automata M_1 – M_6 , where $\mathcal{L}(M'_1) = \text{Sub}(\tilde{x})$, $\mathcal{L}(M_1) = \text{Sub}(\tilde{x})$, $\mathcal{L}(M_2) = \text{Sub}(\tilde{y})$, $\mathcal{L}(M_3) = \mathcal{L}(M_1) \cap \mathcal{L}(M_2)$, $\mathcal{L}(M'_4) = \text{Super}(\tilde{z})$, $\mathcal{L}(M_4) = \text{Super}(\tilde{z})$, $\mathcal{L}(M_5) = \mathcal{L}(M_3) \cap \mathcal{L}(M_4)$, $\mathcal{L}(M_6) = \text{MaxLen}(\mathcal{L}(M_5)) = \text{CLCSub}(\tilde{x}, \tilde{y}, \tilde{z}) = \{aba\}$, for $\tilde{x} = aba[b, c]$, $\tilde{y} = abb[a, c]$ and $\tilde{z} = [a, c][a]$.

3.3 Supersequence Automaton for Degenerate String

In this section, we present a novel type of finite automaton, referred to as the *supersequence automaton* (*SuperA*) henceforth, which accepts the set of all supersequences of a given

degenerate string. The supersequence automaton is deterministic and minimal and its number of states and transitions are linear with respect to the length of the given degenerate string.

The set of all supersequences of degenerate string $\tilde{x} = [\tilde{x}_{11}, \tilde{x}_{12} \dots \tilde{x}_{1|\tilde{x}_1|}] \dots [\tilde{x}_{n1}, \tilde{x}_{n2} \dots \tilde{x}_{n|\tilde{x}_n|}]$ (see Def. 4) can be expressed by the regular expression:

$$R_P = \Sigma^*(\tilde{x}_{11} + \tilde{x}_{12} + \dots + \tilde{x}_{1|\tilde{x}_1|} + \varepsilon) \Sigma^* \dots \Sigma^*(\tilde{x}_{n1} + \tilde{x}_{n2} + \dots + \tilde{x}_{n|\tilde{x}_n|} + \varepsilon) \Sigma^*.$$

The regular expression R_P can be directly used for the construction of a nondeterministic automaton accepting the set $Super(\tilde{x})$ (see Fig. 3.2, automaton M'_4). Let us denote the automaton constructed in this way by M' . The deterministic automaton M obtained by determinization of automaton M' (see Fig. 3.2, automaton M_4) is defined as follows:

$M = (Q, \Sigma, \delta, q_0, F)$, where $Q = \{q_0, q_1, q_2, \dots, q_n\}$, $F = \{q_n\}$ and $\delta(q_{i-1}, s) = q_i$, for all $s \in \tilde{x}_i$, $\delta(q_{i-1}, s) = q_{i-1}$, for all $s \in \{\Sigma \setminus \tilde{x}_i\}$, where $1 \leq i \leq n$, and $\delta(q_n, s) = q_n$ for all $s \in \Sigma$.

The deterministic supersequence automaton M as defined above can be constructed directly without the prior construction of nondeterministic automaton and its consequent determinization. In Appendix C, we provide an online algorithm working in linear time for its direct construction. The basic properties of the supersequence automaton and of the Algorithm 4 for its direct construction can be summarized in the form of following Lemmas.

Lemma 5. *Given a degenerate string $\tilde{x}$ of length n , Algorithm 4 correctly constructs a deterministic finite automaton M accepting language $\mathcal{L}(M) = Super(\tilde{x})$ in $\mathcal{O}(|\Sigma|n)$ time. M has $n + 1$ states and $|\Sigma|(n + 1)$ transitions.*

3.4 Maximum Length Strings Automaton

In this section, we discuss the algorithm for transforming an automaton representing a finite language into an automaton accepting only a subset of strings of that language having maximum length. The algorithm assumes that the input automaton is deterministic and acyclic and it works in linear time with respect to the number of transitions of the input automaton. The resulting automaton is deterministic, acyclic and has at most as many states and transition as the input automaton.

To achieve that the resulting automaton accepts only string of maximum length, we need to remove from the automaton all transitions and states which are not part of any longest path from the initial state to some of final states. Hence, if the original automaton is $M' \leftarrow (Q', \Sigma, \delta', q_0, F')$ then the resulting automaton is $M \leftarrow (Q, \Sigma, \delta, q_0, F)$, where $Q \subseteq Q'$, $F \subseteq F'$ and $\delta \subseteq \delta'$. Since the input automaton is acyclic, to search for the longest paths, we employ a modification of the longest-path algorithm for directed acyclic graphs (DAGs) based on the topological ordering of nodes (states), which works in linear time with respect to a number of edges of a given graph. The details are provided in Appendix D along with the formal steps in Algorithm 5. The basic properties of the algorithm and the resulting automaton can be summarized in the form of following Lemma.

Lemma 6. *Given an acyclic deterministic finite automaton M' with n transitions, Algorithm 5 correctly constructs a deterministic finite automaton M accepting language $\mathcal{L}(M) = MaxLen(\mathcal{L}(M'))$ in $\mathcal{O}(n)$ time. M has at most as many states and at most as many transitions as M' .*

3.5 Complexity

In this section, we present the following lemma which proves the correctness of Algorithm 1 and deduce its time complexity.

Lemma 7. *Given degenerate strings $\tilde{x}$, $\tilde{y}$ and $\tilde{z}$ of length n , m and r , respectively, Algorithm 1 correctly constructs finite automaton M accepting language $\mathcal{L}(M) = CLCSub(\tilde{x}, \tilde{y}, \tilde{z})$ in $\mathcal{O}(|\Sigma|nmr)$ time.*

Proof. Correctness: The algorithm is based on Lemma 1. Individual correctness of the component algorithms in Algorithm 1 follows from the discussions and lemmas in the corresponding sections. Therefore, it remains to show that the preconditions of those Lemmas are guaranteed. According to Lemma 3, the resulting automata M_1 and M_2 are acyclic. Therefore, it follows from Lemma 4 that the automaton M_3 is acyclic too. Since the automaton M_3 is acyclic the automaton M_5 is acyclic as well (Lemma 4). Finally, since automaton M_5 is acyclic, automaton M_6 is acyclic.

Time complexity: Automata M_1 and M_2 are constructed in $\mathcal{O}(|\Sigma|n)$ and $\mathcal{O}(|\Sigma|m)$ time (Lemma 3) and they have at most $n + 1$ and $m + 1$ states, respectively, (Lemma 3). Therefore automaton M_3 is constructed in $\mathcal{O}(|\Sigma|nm)$ time (Lemma 4) and automaton M_3 has at most $(n + 1)(m + 1)$ states. Automaton M_4 has exactly $r + 1$ states (Lemma 5) and is constructed in $\mathcal{O}(|\Sigma|r)$ time (Lemma 5). Thus, automaton M_5 has at most $(n + 1)(m + 1)(r + 1)$ states, $|\Sigma|(n + 1)(m + 1)(r + 1)$ transitions (Lemma 4) and is constructed in $\mathcal{O}(|\Sigma|nmr)$ (Lemma 4). Construction of M_6 from M_5 takes thus $\mathcal{O}(|\Sigma|nmr)$ (Lemma 6). Therefore, the total time complexity of the algorithm is $\mathcal{O}(|\Sigma|nmr)$. $\square$

4 Conclusion

In this paper, we have studied the Constrained Longest Common Subsequence (CLCS) problem for degenerate strings. CLCS is a relatively new variant of the classical Longest Common Subsequence problem and is motivated by bioinformatics. Our approach to solve this problem utilizes subsequence and supersequence automata for degenerate strings. We have presented linear time algorithms for the construction of the above two automata and have used them to solve CLCS problem for degenerate strings in $\mathcal{O}(|\Sigma|nmr)$ time. We believe the subsequence and supersequence automata along with the techniques used in this paper can be employed to different other problems involving degenerate strings.

5 Bibliography

References

1. Chin, F.Y.L., De Santis, A., Ferrara, A.L., Ho, N.L., Kim, S.K.: A simple algorithm for the constrained sequence problems. Inf. Process. Lett. **90** (4) (2004) 175–179
2. Arslan A.N., Egecioğlu Ö.: Algorithms for the constrained longest common subsequence problems. Proceedings of the Prague Stringology Club Workshop '04, Prague, (2004) 24–32 **16** (6)
3. Arslan A.N., Egecioğlu Ö.: Algorithms for the constrained longest common subsequence problems. Int. J. Found. Comput. Sci. **16** (6) (2005) 1099–1109
4. Tsai, Y.T.: The constrained common subsequence problem. Inf. Process. Lett. **88** (2003) 173–176

5. Troníček, Z., Melichar, B.: Directed acyclic subsequence graph. Proceedings of the Prague Stringology Club Workshop '98, Prague,(1998) 107–118
6. Baeza-Yates, R.A.: Searching subsequences. Theoretical Computer Science **78** (1991) 363–376
7. Hébrard, J.J., Crochemore, M.: Calcul de la distance par les sous-mots. RAIRO Inform, Théor. Appl. **20** (1986) 441–456
8. Melichar, B., Holub J., Mužátko P.: Languages and translation. Publishing house of CTU, (1997)

A Algorithm for Construction of Subsequence Automaton for Degenerate Strings

In this section, we present an online algorithm to directly construct the deterministic subsequence automaton M for a given degenerate string $\tilde{x} = \tilde{x}_1\tilde{x}_2\ldots\tilde{x}_n$. The algorithm is an extension of left to right algorithm for constructing DASG presented in [5].

The algorithm builds $SubA$ from left to right, i.e. in each step, $SubA(\tilde{x}_1\tilde{x}_2\ldots\tilde{x}_{k+1})$ is created from $SubA(\tilde{x}_1\tilde{x}_2\ldots\tilde{x}_k)$. This operation is performed by adding a new state q_{k+1} on the left of the automaton and by adding new transitions for all symbols $s \in \tilde{x}_{k+1}$ leading from all states q_ℓ not having transition for s , where $1 \leq \ell \leq k$. The procedure of adding new transitions is based on the following discussion. Firstly, let us consider a prefix $\tilde{x}[1..k] = \tilde{x}_1\tilde{x}_2\ldots\tilde{x}_i\tilde{x}_{i+1}\ldots\tilde{x}_k$ such that $s \in \tilde{x}_i$ and $s \notin \tilde{x}_l$, where $1 \leq i < l \leq k < n$ then it holds $\delta(q_l, s)$ is undefined for $i \leq l \leq k$. Secondly, let us consider extended prefix $\tilde{x}[1..k+1] = \tilde{x}_1\tilde{x}_2\ldots\tilde{x}_i\tilde{x}_{i+1}\ldots\tilde{x}_k\tilde{x}_{k+1}$, where $\tilde{x}_{k+1}$ is such that $s \in \tilde{x}_{k+1}$ then it holds $\delta(q_l, s) = q_{k+1}$ for $i \leq l \leq k$. Therefore it is sufficient to add a transition for the added symbol s only to all states $q_i, q_{i+1}, \ldots, q_k$. This can be easily ensured by memorizing the leftmost state (its index) for each symbol of the alphabet not having the appropriate transition. The steps are formally presented in Algorithm 2.

```

procedure: SUBSEQUENCE-AUTOMATON
input: degenerate string  $\tilde{x} = \tilde{x}_1\tilde{x}_2\ldots\tilde{x}_n$ 
output: subsequence automaton  $M$ ,  $\mathcal{L}(M) = Sub(\tilde{x})$ 

1 begin
2    $Q \leftarrow \{q_0\}$ 
3    $\delta \leftarrow \emptyset$ 
4   for each  $s \in \Sigma$  do  $leftm[s] \leftarrow 0$ 
5   for  $i \leftarrow 1$  to  $n$  do
6      $Q \leftarrow Q \cup \{q_i\}$ 
7     for each  $s \in \tilde{x}_i$  do
8       for  $j \leftarrow leftm[s]$  to  $i - 1$  do
9          $\delta(q_j, s) \leftarrow q_i$ 
10      end for
11       $leftm[s] \leftarrow i$ 
12    end for
13  end for
14   $F \leftarrow Q$ 
15   $M \leftarrow (Q, \Sigma, \delta, q_0, F)$ 
16  return  $M$ 
17 end

```

Algorithm 2: On-line construction of deterministic finite automaton accepting language $Sub(\tilde{x})$

B Algorithm for Construction of Finite Automaton for Intersection of Languages

procedure: INTERSECTION
input: deterministic finite automata $M_1 = (Q^1, \Sigma, \delta^1, q_0^1, F^1)$, $M_2 = (Q^2, \Sigma, \delta^2, q_0^2, F^2)$
output: deterministic finite automaton $M = (Q, \Sigma, \delta, q_0, F)$, $\mathcal{L}(M) = \mathcal{L}(M_1) \cap \mathcal{L}(M_2)$

```

1 begin
2    $q_0 \leftarrow [q_0^1, q_0^2]$ 
3    $Q \leftarrow \{ [q_0^1, q_0^2] \}$ 
4   if  $(q_0^1 \in F^1) \wedge (q_0^2 \in F^2)$  then  $F \leftarrow \{ [q_0^1, q_0^2] \}$ 
5   else  $F \leftarrow \emptyset$ 
6    $C \leftarrow \text{NEW-QUEUE}()$ 
7    $\text{ENQUEUE}(C, [q_0^1, q_0^2])$ 
8   while not  $\text{EMPTY}(C)$  do
9      $[q^1, q^2] \leftarrow \text{DEQUEUE}(C)$ 
10    for all  $s \in \Sigma$  do
11       $[p^1, p^2] \leftarrow [\delta^1(q^1, s), \delta^2(q^2, s)]$ 
12       $\delta([q^1, q^2], s) \leftarrow [p^1, p^2]$ 
13      if  $[p^1, p^2] \notin Q$  then
14         $\text{ENQUEUE}(C, [p^1, p^2])$ 
15         $Q \leftarrow Q \cup \{ [p^1, p^2] \}$ 
16        if  $(p^1 \in F^1) \wedge (p^2 \in F^2)$  then  $F \leftarrow F \cup \{ [p^1, p^2] \}$ 
17      end if
18    end for
19  end while
20  return  $M$ 
21 end
```

Algorithm 3: Construction of an automaton for intersection of languages

C Algorithm for Construction of Supersequence Automaton for Degenerate String

In this section, we present an algorithm to directly construct the deterministic supersequence automaton M for a given degenerate string $\tilde{x} = \tilde{x}_1\tilde{x}_2\ldots\tilde{x}_n$. First, the initial state q_0 is created and it is memorized as the current last state. Then we traverse the string from left to right as follows. Suppose we are considering $\tilde{x}_i$. First a new state q_i is created and it is memorized as the current last state. Next, loop transitions for all symbols of the complement of symbol-set $\tilde{x}_i$ are added to state q_{i-1} and forward transitions leading from state q_{i-1} to state q_i for all symbols of symbol set $\tilde{x}_i$ are created. Finally, loop transition for all symbols of the alphabet are added to the current last state. See Algorithm 4 for details. An example of supersequence automaton constructed for degenerate string $\tilde{z} = [a, c][a]$ is depicted in the Fig.3.2 (automaton M_4).

D Algorithm for Construction of Maximum Length Strings Automaton

In general, the presented algorithm removes from the input automaton some states and some transitions to achieve that the resulting automaton accepts only the longest string of the language of the input automaton. The algorithm proceeds in three stages as described below.

procedure: SUPERSEQUENCE-AUTOMATON
input: a degenerate string $\tilde{x} = \tilde{x}_1\tilde{x}_2\ldots\tilde{x}_n$
output: supersequence automaton M , $\mathcal{L}(M) = \text{Super}(\tilde{x})$

```

1 begin
2    $Q \leftarrow \{q_0\}$ 
3    $last \leftarrow q_0$ 
4   for  $i \leftarrow 1$  to  $n$  do
5      $Q \leftarrow Q \cup \{q_i\}$ 
6      $last \leftarrow q_i$ 
7     for each  $s \in \tilde{x}_i$  do  $\delta(q_{i-1}, s) \leftarrow q_{i-1}$ 
8     for each  $s \in \{\Sigma \setminus \tilde{x}_i\}$  do  $\delta(q_{i-1}, s) \leftarrow q_i$ 
9   end for
10  for each  $s \in \Sigma$  do  $\delta(q_n, s) \leftarrow q_n$ 
11   $M \leftarrow (Q, \Sigma, \delta, q_0, \{q_n\})$ 
12  return  $M$ 
13 end

```

Algorithm 4: Construction of deterministic finite automaton accepting language $\text{Super}(\tilde{x})$

First Stage:

In this stage, the input automaton M accepting language $L = \mathcal{L}(M)$ is transformed into a nondeterministic finite automaton M_R accepting all the longest reversed strings of language L , $\mathcal{L}(M_R) = \text{MaxLen}(L)^R$. Here we employ a modification of the longest-path algorithm for directed acyclic graphs (DAGs) based on the topological ordering of nodes (states). The algorithm keeps for each state q of the automaton M , its input degree $d_{in}[q]$ (number of incoming transitions) and the length of the current longest path to state q from the initial state, i.e. $len[q]$. The algorithm uses queue C for closed states (states with zero input degree). On the beginning, the initial state q_0 of the automaton M is put into queue C . The main loop of the algorithm works as follows: a closed state p is dequeued. Target state q of each transition leading from p is visited, its input degree is decreased by 1 and the length of path from q_0 via p to q is computed. If such a path is the longest one from q_0 to q , p is set as a predecessor of q . Predecessor is memorized in the form of reversed transition. That is why the resulting automaton of this stage accepts reversed longest strings of L . Obviously, if there exist more than one longest path to state q , it has more predecessors. If the input degree of state q is decreased to zero, it is closed and put into queue C . The algorithm keeps a set S of states being targets of the longest paths from state q_0 in graph. This set is actualized during visit of each state. Finally, the states in set S becomes the initial states of the reversed automaton M_R and state q_0 becomes its the only final state.

Second Stage:

In this stage, all unreachable states in automaton M_R are removed by standard procedure (see e.g. [8]) and thus automaton M'_R , $\mathcal{L}(M'_R) = \mathcal{L}(M_R)$ is obtained. This is required because Stage 1 can, in general, produce unreachable states.

Third Stage:

In the third and final stage, automaton M'_R is reversed by standard procedure (see e.g. [8]) into automaton M_M , $\mathcal{L}(M_M) = \text{MaxLen}(L)$.

procedure: MAXLEN-AUTOMATON
input: $M = (Q, \Sigma, \delta, q_0, F)$ - acyclic DFA
output: $M_M = (Q_M, \Sigma, \delta_M, q_0^M, F_M)$, $\mathcal{L}(M_M) = \text{MaxLen}(\mathcal{L}(M))$

```

1 begin
2                                     initialization
3   for all  $q \in Q$  do
4      $d_{in}[q] \leftarrow$  the number of transition leading to  $q$            input degree
5      $len[q] \leftarrow -\infty$                                            length of current longest path from  $q_0$ 
6   end for
7    $len[q_0] \leftarrow 0$ 
8    $S \leftarrow \emptyset$                                                  states with so far maximum length longest path from  $q_0$ 
9    $len_S \leftarrow 0$                                                  length of longest path from  $q_0$  to states in  $S$ 
10
11                                      $M_R$  construction
12    $C \leftarrow \text{NEW-QUEUE}()$                                            queue for closed states,  $d_{in}[q] = 0$ 
13   ENQUEUE( $C, q_0$ )
14   while not EMPTY( $C$ ) do
15      $p \leftarrow \text{DEQUEUE}(C)$ 
16     for all  $s \in \Sigma$  do
17        $q \leftarrow \delta(p, s)$ 
18        $d_{in}[q] \leftarrow d_{in}[q] - 1$ 
19       if  $d_{in}[q] = 0$  then ENQUEUE( $C, q$ )                                $q$  is closed
20       if  $len[q] < len[p] + 1$  then
21          $len[q] \leftarrow len[p] + 1$                                    new longest path from  $q_0$ 
22          $\delta_R(q, s) \leftarrow \{p\}$                                 set the only predecessor
23       else if  $len[q] = len[p] + 1$  then
24          $\delta_R(q, s) \leftarrow \delta_R(q, s) \cup \{p\}$              another longest path from  $q_0$ 
25          $\delta_R(q, s) \leftarrow \delta_R(q, s) \cup \{p\}$              add another predecessor
26       end if
27       if  $q \in F$  then
28         if  $len_S < len[q]$  then
29            $len_S \leftarrow len[q]$                                    new longest path from  $q_0$  in graph
30            $S \leftarrow \{q\}$                                          memorize the only target node
31         else if  $len_S = len[q]$  then
32            $S \leftarrow S \cup \{q\}$                                    another longest path from  $q_0$  in graph
33            $S \leftarrow S \cup \{q\}$                                    add another target node
34         end if
35       end if
36     end for
37   end while
38    $M_R \leftarrow (Q, \Sigma, \delta_R, S, \{q_0\})$ 
39                                     removal of unreachable states
40    $M'_R \leftarrow \text{REMOVE-UNREACHABLE-STATES}(M_R)$ 
41                                     reversal
42    $M_M \leftarrow \text{REVERSE}(M'_R)$ 
43   return  $M_M$ 
44 end

```

Algorithm 5: Construction of deterministic finite automaton M_M for deterministic acyclic finite automaton M accepting language $\mathcal{L}(M_M) = \text{MaxLen}(\mathcal{L}(M))$